Tensor products of Mycielskians of odd cycles with odd wheels and with extended Mycielskians of odd cycles are not word-representable*

Horace Dong
Independent researcher
maxwelldhx@gmail.com
ORCID: 0009-0002-7554-7548

Version 2, 12 November 2026

Abstract

A graph is word-representable if there is a word over its vertex set, containing every vertex, in which two distinct letters alternate exactly when the corresponding vertices are adjacent. Alshammari asked whether the tensor products $\mu(C_{2n+1}) \times W_{2m+1}$ and $\mu'(C_{2n+1}) \times W_{2m+1}$ are non-word-representable for all $n \geq 1$ and $m \geq 2$, where μ and μ' denote the Mycielskian and the extended Mycielskian and W_{2m+1} is the wheel on an odd cycle, and whether the same holds for $\mu(C_{2n+1}) \times \mu'(C_{2m+1})$ and $\mu'(C_{2n+1}) \times \mu'(C_{2m+1})$ when $1 \leq n < m$. We show that the answer to all three questions is yes. For the wheels, each product contains an induced copy of $\mu(C_{2k+1})$ or $\mu'(C_{2k+1})$ for some $k \geq 2$, and these graphs are not word-representable by theorems of Hameed and of Kitaev and Pyatkin. For the products with $\mu'(C_{2m+1})$ we fold the longer cycle onto the shorter one: the root together with the top vertices and shadows of any three consecutive columns of the fold induces one of two seven-vertex graphs, and a colouring of the orientations of five edges, checked by computer, must change at each step around an odd cycle. All the results, and the facts their proofs use, are formally verified in Lean 4. The proof was found by AI agents working under the author's direction; their role is described at the end of the paper.

1 Introduction

All graphs are finite and simple. Two distinct letters x and y *alternate* in a word w if deleting from w all letters other than x and y leaves a word $xyxy\cdots$ or $yxyx\cdots$, of even or odd length. A graph G is *word-representable* if there is a word w over the alphabet $V(G)$ in which every vertex occurs and, for all distinct vertices x and y, the letters x and y alternate in w if and only if xy is an edge [6, 1]. Halldórsson, Kitaev and Pyatkin [4] proved that a graph is word-representable if and only if it has a semi-transitive orientation.

The *tensor product* $G \times H$ of graphs G and H has vertex set $V(G) \times V(H)$, and (a, b) is adjacent to (a', b') if and only if aa' is an edge of G and bb' is an edge of H. Kitaev and Lozin asked whether anything definite can be said about the word-representability of tensor products of two word-representable graphs, of a word-representable and a non-word-representable graph, or of two non-word-representable graphs [6, Problem 7.2.5], as quoted in [1, Problem 1.1]. Choi, Kim and Kim [2] showed that the tensor product of two graphs with semi-transitive orientations need not have one. Alshammari [1] studied tensor products of graphs containing

* *Version 2.* Version 1 (September 2026) answered Problem 5.1 of [1]; version 2 adds Theorem 3, which answers Problems 5.2 and 5.3, with its formal verification. Theorem 1 and its proof are unchanged.

wheels, complete graphs and (extended) Mycielskians of cycles, and posed four problems. The first of them is the following [1, Problem 5.1].

> Is it true that $\mu_{2n+1} \times W_{2m+1}$ and $\mu'_{2n+1} \times W_{2m+1}$ is always non-word-representable for $n \geq 1$ and $m \geq 2$?

Here, as in [1], W_k is the wheel obtained from the cycle C_k by adding a vertex adjacent to all its vertices, $\mu_k = \mu(C_k)$ is the Mycielskian of C_k and $\mu'_k = \mu'(C_k)$ its extended Mycielskian (Section 2). We answer the question in the affirmative.

Theorem 1. *For all integers $n \geq 1$ and $m \geq 2$, the graphs $\mu_{2n+1} \times W_{2m+1}$ and $\mu'_{2n+1} \times W_{2m+1}$ are not word-representable.*

The proof (Section 3) finds a non-word-representable induced subgraph: for $n \geq m$, a copy of the first factor, embedded through a homomorphism into W_{2m+1}, as in the proof of [1, Theorem 3.6]; for $n \leq m$, a copy of μ_{2m+1}, by an embedding that adapts the one in the proof of [1, Theorem 4.3]. That these subgraphs are not word-representable is due to Hameed [5] and to Kitaev and Pyatkin [7]. The next two problems of [1] are the following [1, Problems 5.2 and 5.3].

> Is it true that $\mu_{2n+1} \times \mu'_{2m+1}$ is always non-word-representable for $n < m$?
> Is it true that $\mu'_{2n+1} \times \mu'_{2m+1}$ is always non-word-representable for $n < m$?

Here $n \geq 1$, so that C_{2n+1} is a cycle. Version 2 of this paper also answers these questions in the affirmative (Theorem 3: for all $1 \leq n < m$, both graphs are not word-representable); the numbering of the results of version 1 is kept, and the section on formal verification is now Section 5.

The proof (Section 4) does not find a known non-word-representable induced subgraph. It folds the cycle C_{2m+1} onto C_{2n+1} as in Section 3, and shows that the root together with the top vertices and shadows of any three consecutive columns of the folded copy induces one of two seven-vertex graphs. A computation over the orientations of the edges of these two graphs, carried out by the Lean kernel, then shows that a word representing the product would give a two-colouring that changes at each step around the odd cycle C_{2m+1}, which is impossible. The same kind of computation is the core of the Lean proof of Theorem 2 below. We claim nothing about the last problem of [1]: whether the tensor product of two non-word-representable graphs can be word-representable [1, Problem 5.4] remains open.

Prior work. We found no earlier answer to [1, Problem 5.1]. We searched on 27 September 2026 between 18:06 and 19:02 UTC: the arXiv (its API, for the topic and the author); the works citing [1] in OpenAlex (there were none); MathDB; MathOverflow and Mathematics Stack Exchange, through the Stack Exchange API; GitHub (code, commits, issues, pull requests and repositories), including the repositories `trureturing` [10] and `SCOPE2026` [9] of AI-assisted work on open problems; Wikipedia; and web search. A second AI system repeated a smaller search later that day. A final check between 23:38 UTC on 28 September and 00:17 UTC on 29 September 2026 (the arXiv, including new versions of [1]; the works citing it in Semantic Scholar and OpenAlex; MathOverflow and Mathematics Stack Exchange; GitHub; MathDB; and web search) found nothing new. We did not search Google Scholar, Scopus or Web of Science, and we did not consult the book [6] itself. This reports the coverage of our searches, not a guarantee of priority.

For Problems 5.2 and 5.3, we found no earlier answer, claim or announced work in a search on 9 November 2026 between 22:11 and 22:27 UTC: the arXiv (only version 1 of [1]; its API, for the topic and the author); the works citing [1] in OpenAlex and Semantic Scholar (there were none); Crossref and Zenodo; GitHub (issues, pull requests, commits and repositories), including

`trureturing` [10], where an issue opened on 27 October 2026 concerns only Problem 5.4 and calls Problems 5.2 and 5.3 open, and other repositories of AI-assisted work on open problems; MathDB; MathOverflow and Mathematics Stack Exchange; and web search. Google Scholar could not be searched, and Semantic Scholar returned errors for the metadata of [1]. A further search on 11 November 2026 (14:36–14:46 UTC) found nothing new. This reports the coverage of our searches, not a guarantee of priority.

2 Definitions and three known facts

For $k \geq 3$ the cycle C_k has vertex set $\mathbb{Z}_k$, with i adjacent to $i \pm 1$. The wheel W_k has the *rim* vertices r_i, $i \in \mathbb{Z}_k$, which form a copy of C_k, and the *hub* h, adjacent to every rim vertex. The Mycielskian μ_k has the *top* vertices v_i, the *shadows* u_i ($i \in \mathbb{Z}_k$) and the *root* w; its edges are v_iv_{i+1}, u_iv_{i+1}, u_iv_{i-1} and wu_i for $i \in \mathbb{Z}_k$. The extended Mycielskian μ'_k has the same vertices and the edges v_iv_{i+1}, u_iv_j for all $j \neq i$, and wu_i [1, 7]. In both graphs the shadows are pairwise non-adjacent, and the root is adjacent to no top vertex.

Lemma 2.1 (well known; see [6])**.** *An induced subgraph of a word-representable graph is word-representable.*

Proof. If w represents G and $S \subseteq V(G)$, delete from w all letters outside S. Every letter of S still occurs, and whether two letters of S alternate depends only on the subword formed by these two letters, which does not change; so the new word represents the subgraph induced by S. □

Lemma 2.2. *If $f\colon G \to H$ is a homomorphism of graphs, then $x \mapsto (x, f(x))$ is an isomorphism of G onto an induced subgraph of $G \times H$.*

Proof. The map is injective. For vertices x and y of G, the pair $(x, f(x))$, $(y, f(y))$ is an edge of $G \times H$ if and only if xy is an edge of G and $f(x)f(y)$ is an edge of H; as f is a homomorphism, this holds if and only if xy is an edge of G. □

Theorem 2 (Hameed [5]; Kitaev and Pyatkin [7, Theorem 4 and Remark 5])**.** *For every $t \geq 1$, the graphs μ_{2t+1} and μ'_{2t+1} are not word-representable.*

Hameed [5] proved that μ'_{2t+1} has no semi-transitive orientation; Kitaev and Pyatkin [7, Theorem 4] proved the same for μ_{2t+1}, and observed that their proof also applies to μ'_{2t+1} [7, Remark 5]. By [4], these graphs are therefore not word-representable. We use Theorem 2 only for $t \geq 2$.

3 Proof of Theorem 1

For odd integers $K \geq k \geq 3$ define the *fold* $\varphi_{K,k}\colon \mathbb{Z}_K \to \mathbb{Z}_k$ by $\varphi_{K,k}(i) = i$ for $0 \leq i \leq k-1$, and, for $k \leq i \leq K-1$, by $\varphi_{K,k}(i) = k-2$ if $i-k$ is even and $\varphi_{K,k}(i) = k-1$ if $i-k$ is odd. For $K = k$ it is the identity. For $K > k$ it is a homomorphism $C_K \to C_k$: the images of i and $i+1$ are adjacent for $0 \leq i \leq K-2$, and $\varphi_{K,k}(K-1) = k-1$ is adjacent to $\varphi_{K,k}(0) = 0$, as $K-1-k$ is odd. In particular $\varphi_{K,k}(i) \neq \varphi_{K,k}(i \pm 1)$ for every i.

Lemma 3.1. *Let $k \geq K \geq 5$ be odd. There are homomorphisms $\mu_k \to W_K$ and $\mu'_k \to W_K$.*

Proof. Put $\psi = \varphi_{k,K}$. Map v_i and u_i to $r_{\psi(i)}$, and w to h. The edges v_iv_{i+1} and $u_iv_{i\pm1}$ of μ_k go to the rim edges $r_{\psi(i)}r_{\psi(i+1)}$ and $r_{\psi(i)}r_{\psi(i\pm1)}$, and wu_i goes to the edge $hr_{\psi(i)}$. For μ'_k, map v_i to $r_{\psi(i)}$, every u_i to h, and w to r_0: the edges v_iv_{i+1} go to rim edges, and the edges u_iv_j and wu_i to edges at the hub. □

Lemma 3.2. *Let $K \geq k \geq 3$ be odd, with $K \geq 5$, and let G be μ_k or μ'_k. Then μ_K is isomorphic to an induced subgraph of $G \times W_K$.*

Proof. Put $\varphi = \varphi_{K,k}$ and define ι on the vertices of μ_K by

$$\iota(v_i) = (v_{\varphi(i)}, r_i), \qquad \iota(u_i) = (u_{\varphi(i)}, r_i), \qquad \iota(w) = (w, h).$$

It is injective: the second coordinate determines i, or shows that the vertex is w, and the first coordinate tells top vertices from shadows.

Edges go to edges. For v_iv_{i+1}: the top vertices $v_{\varphi(i)}$ and $v_{\varphi(i+1)}$ are adjacent in G, as φ is a homomorphism, and r_ir_{i+1} is an edge. For $u_iv_{i\pm1}$: in μ_k the shadow $u_{\varphi(i)}$ is adjacent to $v_{\varphi(i\pm1)}$ because $\varphi(i)$ and $\varphi(i \pm 1)$ are adjacent in C_k, and in μ'_k because they are different; and $r_ir_{i\pm1}$ is an edge. For wu_i: w is adjacent to $u_{\varphi(i)}$ in G, and h to r_i.

Non-edges go to non-edges. The non-edges of μ_K are v_iv_j with $j \notin \{i-1, i, i+1\}$, u_iv_j with $j \notin \{i-1, i+1\}$ (including $j = i$), u_iu_j with $j \neq i$, and wv_j. In the first two cases the second coordinates r_i and r_j are not adjacent in W_K, as $j \notin \{i-1, i+1\}$ and W_K has no loops. For u_iu_j the first coordinates are two shadows, or one shadow twice, which are not adjacent in G; for wv_j the first coordinates w and $v_{\varphi(j)}$ are not adjacent in G. So ι is an isomorphism onto the subgraph of $G \times W_K$ induced by its image. □

Proof of Theorem 1. Let $k = 2n+1$ and $K = 2m+1 \geq 5$, and let G be μ_k or μ'_k. If $n \geq m$, Lemmas 3.1 and 2.2 give an induced copy of G in $G \times W_K$. If $n \leq m$, Lemma 3.2 gives an induced copy of μ_K in $G \times W_K$. In both cases the copy is not word-representable by Theorem 2, since $k \geq K \geq 5$ in the first case; so $G \times W_K$ is not word-representable by Lemma 2.1. □

4 Problems 5.2 and 5.3

Theorem 3. *For all integers $1 \leq n < m$, the graphs $\mu_{2n+1} \times \mu'_{2m+1}$ and $\mu'_{2n+1} \times \mu'_{2m+1}$ are not word-representable.*

The two windows. Let F_9 be the graph on the seven vertices $w, v_1, v_2, v_3, u_1, u_2, u_3$ with the nine edges

$$wu_1,\ wu_2,\ wu_3,\ v_1v_2,\ v_2v_3,\ u_1v_2,\ u_2v_1,\ u_2v_3,\ u_3v_2,$$

and let F_{11} be F_9 with the two further edges u_1v_3 and u_3v_1. For $K \geq 5$, the subgraph of μ_K induced by $w, v_i, v_{i+1}, v_{i+2}, u_i, u_{i+1}, u_{i+2}$ is F_9, and that of μ'_K is F_{11} (with v_j, u_j in the roles of v_{j-i+1}, u_{j-i+1}).

Let a word x represent a graph G, and orient each edge of G from the end vertex that occurs first in x. Section 5 shows that this orientation has the following property (P): if $a \to c \to d \to b$ is a directed path and a is adjacent to b, then a is adjacent to d and c to b. Given seven distinct vertices of G inducing F_9 or F_{11} in the order above, call the orientation of the five edges $wu_1, wu_2, v_1v_2, v_1u_2, u_1v_2$ the *input state*, and that of the five edges $wu_2, wu_3, v_2v_3, v_2u_3, u_2v_3$ the *output state*; each is one of 2^5 orientations of five edges, read in this order.

Lemma 4.1 (computer-checked)**.** *There is a colouring χ of the 2^5 states with two colours such that, for every graph G, every orientation of G with property (P), and every seven distinct vertices of G inducing F_9 or F_{11} in the order above, the input state and the output state have different colours.*

Only the edges inside the seven vertices matter: as the vertices induce F_9 or F_{11}, every directed path among them whose end vertices are adjacent has its two chords among the edges of F_9 or F_{11}. The Lean kernel checks the lemma for F_9 by running through all 2^9 orientations of its edges and for F_{11} through all 2^{11}, with one fixed colouring χ; the colouring is the one

used for F_9 in version 1. We do not know a proof of Lemma 4.1 without this computation. The check for F_9 is the one used in the Lean proof of Theorem 2 (Section 5); the check for F_{11} is new, and it uses the colouring of F_9, whereas the Lean proof of Theorem 2 for μ' uses another one. In Lean the lemma is stated for the orientation given by a word; the finite check covers every orientation of the edges of F_9 or F_{11} with property (P).

The folded windows. Let $k = 2n+1$ and $K = 2m+1$, so that $3 \le k < K$ and $K \ge 5$; let H be μ_k or μ'_k, and $G = H \times \mu'_K$. Put $\varphi = \varphi_{K,k}$, the fold of Section 3, and, for $i \in \mathbb{Z}_K$,

$$T_i = (v_{\varphi(i)}, v_i), \qquad S_i = (u_{\varphi(i)}, u_i), \qquad R = (w, w).$$

Lemma 4.2. *For every $i \in \mathbb{Z}_K$, the seven vertices $R, T_i, T_{i+1}, T_{i+2}, S_i, S_{i+1}, S_{i+2}$ of G are distinct and induce F_9 or F_{11}, in the roles of $w, v_1, v_2, v_3, u_1, u_2, u_3$.*

Proof. The second coordinates are distinct, since $i, i+1, i+2$ are distinct in $\mathbb{Z}_K$ for $K \ge 5$. Recall that in μ'_K the root is adjacent exactly to the shadows, v_j to $v_{j\pm1}$, and v_j to u_l exactly when $j \ne l$, and that shadows are pairwise non-adjacent. We go through the pairs. *The root and a shadow*: w and u are adjacent in both factors, so RS_j is an edge. *The root and a top vertex, two shadows, a top vertex and the shadow of the same column*: the second coordinates are not adjacent, so these are not edges. *Top vertices of consecutive columns*: $v_j v_{j+1}$ is an edge of μ'_K, and $v_{\varphi(j)}v_{\varphi(j+1)}$ one of H, as φ is a homomorphism. *The top vertices T_i, T_{i+2}*: v_i and v_{i+2} are not adjacent in C_K. *A top vertex and a shadow of consecutive columns*: the second coordinates have different indices, so they are adjacent; the first coordinates $v_{\varphi(j)}$ and $u_{\varphi(j\pm1)}$ are adjacent in μ_k, as $\varphi(j)$ and $\varphi(j \pm 1)$ are adjacent in C_k, and in μ'_k, as they are different. *The pairs T_iS_{i+2} and $T_{i+2}S_i$*: the second coordinates are adjacent; the first coordinates are adjacent in μ_k if and only if $\varphi(i)$ and $\varphi(i+2)$ are adjacent in C_k, and in μ'_k if and only if they are different. Both conditions are symmetric in i and $i+2$, so both pairs are edges or neither is. In the first case the seven vertices induce F_{11}, in the second F_9. □

Proof of Theorem 3. Suppose that a word represents G, orient its edges as above, and let c_i be the colour χ of the input state of the window $W_i = (R, T_i, T_{i+1}, T_{i+2}, S_i, S_{i+1}, S_{i+2})$ of Lemma 4.2. The output state of W_i is the orientation of the edges RS_{i+1}, RS_{i+2}, $T_{i+1}T_{i+2}$, $T_{i+1}S_{i+2}$, $S_{i+1}T_{i+2}$, which is the input state of W_{i+1}. By Lemmas 4.1 and 4.2, $c_{i+1} \ne c_i$ for every $i \in \mathbb{Z}_K$. Going once around $\mathbb{Z}_K$ changes the colour K times, an odd number, so $c_0 \ne c_0$, a contradiction. □

The windows of different i may be of different types, and nothing is claimed about pairs of vertices of the folded copy that lie in no common window; the folded copy need not be an induced copy of μ_K or μ'_K.

5 Formal verification

The results of this paper are formally verified in Lean 4 (version 4.34.1) with Mathlib [8] (version v4.34.1, commit `d13f23b7`).

Statements before proofs. After the proof had been found, the agent that found it wrote a Lean file of definitions and statements, the *statement file*, which contains no proofs. An independent agent reviewed version 1 of this file against the sources and tested its definitions with programs of its own; version 2 applies its corrections and adds the lemmas used in the proof. A second AI system reviewed version 2, and it was frozen on 27 September 2026 at 22:04 UTC by recording its SHA-256 hash

`40c8efd69f8499c9874f07163043c0793c94c8597b7a3902d5aeb0a2ce0c9b58`.

The proofs were written after the freeze, in separate files that import the statement file, which was not changed.

For Theorem 3, a second statement file (`TaskF/Challenge.lean`) states the two answers, using the definitions of the first file, and a file of internal statements (`TaskF/RouteContract.lean`) states the steps of the proof: Lemma 4.2 for each of the two products, Lemma 4.1 for one fixed colouring, the finite check behind it and the agreement of the states of consecutive windows. A Codex agent that had not written them reviewed both files against [1], and they were frozen on 9 November 2026 at 23:14 UTC by recording their SHA-256 hashes

`aef837f367e24d4d1dc10dd2e0be6b4aac2c5040392b00b607fa981717d9c0b5`,
`f3e39eff93c2c46026fc241b0f119453f5a3f361099b484169ea24ca5bc99553`.

The written proof of Section 4 and the Lean proofs were written after this freeze; the approach had been chosen before it. The files of version 1 were not changed.

What is proved. Table 1 lists the eleven statements of the first frozen file and the two of the second; the file of checks of version 1 and the module `TaskF/Main.lean` prove each of them. In ASCII notation, the definition of word-representability and the statement of Theorem 1 for μ read as follows (implicit arguments omitted), with `forall`, `exists`, `->`, `<->`, `/\`, `in`, `!=` and `Not` for $\forall$, $\exists$, $\to$, $\leftrightarrow$, $\wedge$, $\in$, $\neq$ and $\neg$.

```
def Alternate (w : List V) (x y : V) : Prop :=
  noRepeat (w.filter (fun z => z == x || z == y)) = true
def Represents (G : SimpleGraph V) (w : List V) : Prop :=
  (forall v : V, v in w) /\
    forall x y : V, x != y -> (G.Adj x y <-> Alternate w x y)
def WordRepresentable (G : SimpleGraph V) : Prop := exists w : List V, Represents G w
def Problem1Mu : Prop := forall n m : Nat, 1 <= n -> 2 <= m ->
  Not (WordRepresentable (tensorProd (mu (2 * n + 1)) (wheel (2 * m + 1))))
```

Here `noRepeat` says that no two consecutive letters are equal, and `mu n` and `muExt n` stand for `mycielskian (cycleGraph n)` and `extMycielskian (cycleGraph n)`. The name `Problem1Mu` comes from the label of [1, Problem 5.1] in the source of [1]. The statements of Theorem 3 read

```
def Problem5_2 : Prop := forall n m : Nat, 1 <= n -> n < m ->
  Not (WordRepresentable (tensorProd (mu (2 * n + 1)) (muExt (2 * m + 1))))
def Problem5_3 : Prop := forall n m : Nat, 1 <= n -> n < m ->
  Not (WordRepresentable (tensorProd (muExt (2 * n + 1)) (muExt (2 * m + 1))))
```

Lemmas 2.1, 2.2, 3.1 and 3.2 are proved in Lean as in the text.

How Theorem 2 is proved in Lean. The Lean proof does not follow [5, 7]; it works with words directly. Let $x \neq y$ be letters of a word, and let x occur first. Then x and y alternate if and only if every prefix of the word contains at least as many letters x as letters y, and at most one more. Now let a word represent a graph, and orient each edge from the end vertex that occurs first. Write $\#x$ for the number of letters x in a prefix. If $a \to c \to d \to b$ is a directed path and a is adjacent to b, then a occurs before b, so in every prefix each of $\alpha = \#a - \#c$, $\beta = \#c - \#d$, $\gamma = \#d - \#b$ and $\delta = \#a - \#b$ is 0 or 1. Hence $\#a - \#d = \alpha + \beta = \delta - \gamma$ and $\#c - \#b = \beta + \gamma = \delta - \alpha$ are 0 or 1; as a occurs before d, and c before b, the vertex a is adjacent to d, and c to b. For μ_{2t+1} and μ'_{2t+1} with $t \geq 2$, consider the seven vertices $w, v_i, v_{i+1}, v_{i+2}, u_i, u_{i+1}, u_{i+2}$. A computation checked by the Lean kernel runs through all orientations of the edges among them (2^9 for μ, 2^{11} for μ') and shows that in every orientation with the property just proved, the orientation of the five edges between the columns i and $i+1$ (the edges v_iv_{i+1}, v_iu_{i+1}, u_iv_{i+1}, wu_i and wu_{i+1}) and that of the five corresponding edges between the columns $i+1$ and $i+2$ get different colours, for a fixed colouring of the 2^5 orientations of five edges with two colours. Going once around the odd cycle changes the colour an odd number of times, which is impossible. The graphs μ_3 and μ'_3 are equal, and a computation over the 2^{12} orientations of their twelve edges shows that none has the property.

Lean statement	what it says	here
`Problem1Mu`, `Problem1MuExt`	$\mu_{2n+1} \times W_{2m+1}$, resp. $\mu'_{2n+1} \times W_{2m+1}$, is not word-representable for all $n \geq 1$, $m \geq 2$	Thm 1
`Problem5_1`	the conjunction of the two	Thm 1
`MycielskiOddCycleNotWR`, `ExtMycielskiOddCycleNotWR`	μ_{2t+1}, resp. μ'_{2t+1}, is not word-representable for all $t \geq 1$	Thm 2
`WordRepresentableOfEmbedding`	a graph that embeds as an induced subgraph into a word-representable graph is word-representable	Lem. 2.1
`EmbeddingOfHom`	a homomorphism $G \to H$ gives an embedding of G into $G \times H$ as an induced subgraph	Lem. 2.2
`Lemma1Mu`, `Lemma1MuExt`	a homomorphism $\mu_{2n+1} \to W_{2m+1}$, resp. $\mu'_{2n+1} \to W_{2m+1}$, for $2 \leq m \leq n$	Lem. 3.1
`Lemma2Mu`, `Lemma2MuExt`	an embedding of μ_{2m+1} as an induced subgraph into $\mu_{2n+1} \times W_{2m+1}$, resp. $\mu'_{2n+1} \times W_{2m+1}$, for $1 \leq n \leq m$, $m \geq 2$	Lem. 3.2
`Problem5_2`, `Problem5_3`	$\mu_{2n+1} \times \mu'_{2m+1}$, resp. $\mu'_{2n+1} \times \mu'_{2m+1}$, is not word-representable for all $1 \leq n < m$	Thm 3

Table 1: The eleven statements of the first frozen statement file (namespace `WordRepTensor.Challenge`) and the two of the second (`WordRepTensor.TaskF.Challenge`). Each is proved; the proofs of the last two are `WordRepTensor.TaskF.Proof.check_Problem5_2` and `check_Problem5_3`.

Checks. For every statement of Table 1, the axiom audits (`#print axioms`) report only the axioms `propext`, `Classical.choice` and `Quot.sound`. A static scan found no `sorry`, `admit`, `native_decide`, `bv_decide`, `implemented_by`, `extern`, `unsafe` or `opaque`, and no `axiom` declaration, in the proof files. The Lean package of this paper, 11 modules (the statement file, its tests, eight proof modules and the file of checks of version 1), was built module by module in an empty directory, reusing only the pinned toolchain and the compiled Mathlib and its dependencies, and every module was checked with `leanchecker`, all with exit code 0, twice (on 28 September 2026 between 20:12 and 20:21 UTC). For the release, the whole Lean project of the repository was rebuilt from a clean directory and all its modules were checked again with `leanchecker` (28 September 2026, 20:48–22:16 UTC). The Lean files are in version 1.19.0 of the repository [3], directory `lean/Research/WordRepTensor/`.

For Theorem 3, the six new modules (the second statement file, the file of internal statements, three proof modules and a file of checks), together with the eleven of version 1, were built module by module in two new empty directories on the author's Windows computer on 9 November 2026 (finished at 23:42 UTC), and every module was checked with `leanchecker`: all 68 commands ended with exit code 0, with no warning, and the compiled `.olean` files of the two directories are byte for byte identical. The axiom audits of `check_Problem5_2` and `check_Problem5_3` report exactly `propext`, `Classical.choice` and `Quot.sound`. The finite check of Lemma 4.1 for F_{11} is proved with `decide +kernel`, which the Lean kernel evaluates; the static scan above found none of the listed constructs and no `axiom` declaration in the six new modules. From 10 to 11 November 2026 the whole Lean project of the repository was rebuilt from a new empty directory on the author's Windows computer, one module at a time and not with `lake`: all 3011 modules compiled, and 3010 passed `leanchecker`; the remaining one, the root of the project, has only imports and no constants. Some modules of another paper had to be run again after memory failures; no Lean check rejected a module. The module `TaskF/FinalAudit.lean` also checks the instances $(n, m) = (1, 2)$ and $(2, 25)$ of both statements.

Definitions the reader must trust. Apart from Mathlib's graphs, cycle graphs (`cycleGraph`), homomorphisms, embeddings and lists, the statements use the definitions of alternation, representation and word-representability shown above; the tensor product

(`tensorProd`, built with Mathlib's `SimpleGraph.fromRel`); the wheel (`wheel`, on `Sum (Fin n) Unit`); and the Mycielskian and the extended Mycielskian (`mycielskian`, `extMycielskian`, on `Sum V (Sum V Unit)`, with top vertices, shadows and root in this order). A file of known-answer tests checks these definitions on small graphs: adjacencies and edge counts of cycles, wheels, Mycielskians and tensor products; words that represent C_4, C_5, K_3 and $K_2 \times K_2$; words that do not represent a given graph, including words that omit a vertex; small instances of the maps of Lemmas 2.2, 3.1 and 3.2; and a wrong map that is not an embedding.

Faithfulness and non-claims. A word is a list of vertices in which every vertex must occur, as in [1]; so the empty word represents no graph with a vertex. The Lean statements concern word-representability itself, not semi-transitive orientations, so the theorem of [4] is not needed as a hypothesis. Theorem 2 is formalized in its word-representability form; we do not formalize the characterizations of [5, 7] for arbitrary graphs. The internal statements of Theorem 3 are not results of the paper; only `Problem5_2` and `Problem5_3` are claimed. That the Lean definitions agree with those of [1] was checked by the reviewing agents, not by a machine.

How this paper was produced

This work was carried out from 27 to 29 September 2026 (UTC) in a workflow directed by the author, using AI agents. Agents of Anthropic Claude Code, with the model Claude Opus 5.5, selected the question, found the proof, checked it with two separately written programs for small parameters, wrote the statement file of Section 5, carried out the main literature search described in the introduction, and drafted this text. An independent Claude Code agent reviewed version 1 of the statement file against the sources and checked the definitions with its own programs; its corrections were applied in version 2. OpenAI Codex (desktop application; models `gpt-6-astra` and `gpt-6-sol`) then checked the mathematics independently, recomputing the embeddings with its own program, reviewed version 2 of the statement file and froze it. When Codex's usage quota ran out, Claude Code wrote the Lean proofs and ran the checks of Section 5, and an independent Claude Code agent reviewed the proofs. Another independent Claude Code agent refereed a draft of this paper. Codex did not review the Lean proofs of version 1 or version 1 of this paper; the final review of version 1 before its release was done by an independent Claude Code agent in its place.

Version 2 was produced from 9 to 12 November 2026. A Claude Code agent searched the literature for Problems 5.2 and 5.3 on 9 November (22:11–22:27 UTC). OpenAI Codex (desktop application; model `gpt-6.1-sol`) chose the approach, wrote the second statement file and the file of internal statements, which a Codex agent that had not written them reviewed before they were frozen, then wrote the proof of Theorem 3 and the Lean proofs and ran the two builds; a Codex agent that had not written the proofs reviewed the result (9 November 2026, 23:48 UTC). Claude Code (model Claude Opus 5.5) wrote Section 4 from Codex's notes and the Lean files, drafted the other changes of version 2 and rebuilt the whole Lean project of the repository. An independent Claude Code agent (model Claude Fable 5.1), which had written none of the files, reviewed a draft of version 2 against the Lean files, Codex's records and [1] on 10 November 2026, without running Lean; it found no error in the mathematics, and its corrections were applied. Before the release a second such agent (same model) reviewed version 2 again, with the Lean package and the records of the builds, and its corrections were applied. Throughout, "we" refers to this workflow. The author is not a professional mathematician. The author has read version 1 of the paper and, using a side-by-side table, compared the statements of the Lean theorems with the results stated there. The author has read version 2 and, using a side-by-side table, compared the statements of the new Lean theorems with Theorem 3. No human mathematician has reviewed this paper. The correctness of the results rests on the Lean proofs of the statements in Table 1 and on the agreement of those statements with the

definitions of [1], which was checked by the reviewing agents; the written proofs are given for the reader. AI systems are not authors of this paper; the author takes full responsibility for its content.

Thanks are due to Nawaf Alshammari for the questions studied here.

References

[1] N. Alshammari, On the word-representability of tensor product graphs, arXiv:2609.20881v1, 16 September 2026; we use its numbering. As of 9 November 2026 it had only version 1.

[2] I. Choi, J. Kim and M. Kim, On operations preserving semi-transitive orientability of graphs, *J. Comb. Optim.* **37** (2019), 1351–1366, https://doi.org/10.1007/s10878-018-0358-7; arXiv:1810.11620.

[3] H. Dong, *ai4math-results*, repository, https://github.com/Horace-Maxwell/ai4math-results, version 1.19.0 (2026); archived at Zenodo under the concept DOI https://doi.org/10.5281/zenodo.22970254.

[4] M. M. Halldórsson, S. Kitaev and A. Pyatkin, Semi-transitive orientations and word-representable graphs, *Discrete Appl. Math.* **201** (2016), 164–171, https://doi.org/10.1016/j.dam.2015.07.033.

[5] H. Hameed, On semi-transitivity of (extended) Mycielski graphs, *Discrete Appl. Math.* **359** (2024), 83–88, https://doi.org/10.1016/j.dam.2024.07.028.

[6] S. Kitaev and V. Lozin, *Words and Graphs*, Monographs in Theoretical Computer Science, an EATCS Series, Springer, 2015, https://doi.org/10.1007/978-3-319-25859-1.

[7] S. Kitaev and A. Pyatkin, A note on Hameed's conjecture on the semi-transitivity of Mycielski graphs, *Discuss. Math. Graph Theory* **45** (2025), https://doi.org/10.7151/dmgt.2575; quoted from arXiv:2408.05066v1 ("A note on semi-transitivity of Mycielski graphs"), whose numbering we use.

[8] The mathlib Community, The Lean mathematical library, in *Proc. 9th ACM SIGPLAN Int. Conf. Certified Programs and Proofs* (CPP '20), ACM, 2020, https://doi.org/10.1145/3372885.3373824.

[9] SCOPE-Science, `SCOPE2026` repository, https://github.com/SCOPE-Science/SCOPE2026, accessed 27 September 2026.

[10] The Omega Institute, `trureturing` repository, https://github.com/the-omega-institute/trureturing, accessed 27 September 2026.